

Doku

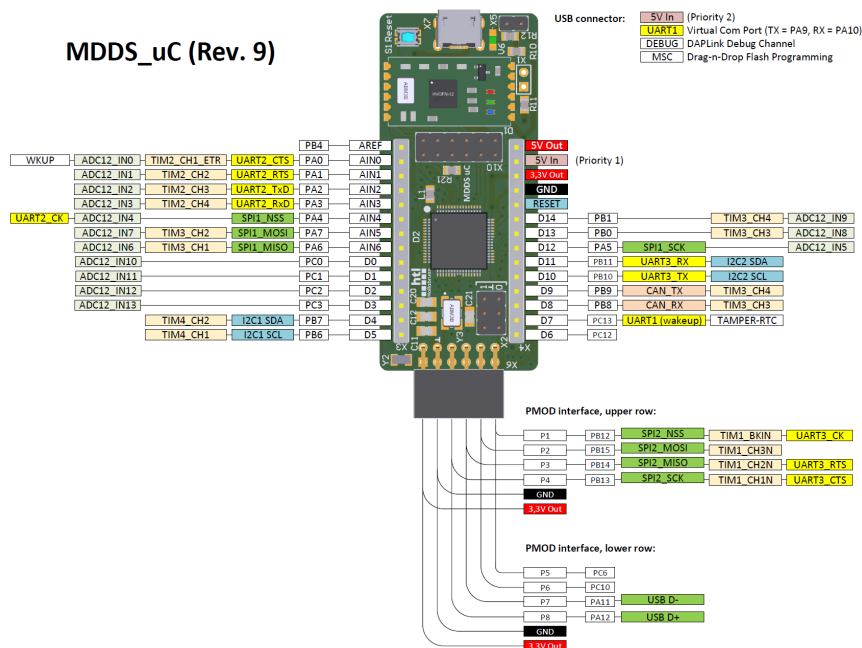
Spannungsversorgung

Für verschiedenste ics oder uC werden LDOS gewählt weil nirgends auf der PCB große kontinuierliche ströme gebraucht werden. es werden 3,3V 5V und eine $\pm$ versorgung benötigt für die benötigten opvs. da die digital versorgung muss nicht schön sein da eigentlich nur spannung gebraucht wird nichts schönen doch die analog 5v versorgung sollte schön sein. es werden hauptsächlich auf Psrr und auf low noise, nicht unbedingt high accuracy.

Drivers

PINs.c

im PINs.c werden die GPIO pins konfiguriert.



WIH 2020-06-30

uC_pinning.png

Der Port A: pin 1,2,4 werden als Output push pull konfiguriert. PA1 ist die CS* für den DAC (AD5689) beim SPI bus. PA2 ist der configurations pin des DAC der zwischen einem gain von 1 und gain von 2 umschaltet, bei einem 1 auf PA2 ist der Gain 2, und damit ist die ausgangs range von 0V bis 5V, und bei PA2 low ist die range 0V bis 2.5V. PIN PA4 ist der CS* für den SPI bus des ADC ADS8681. Port A: 5,6,7: Pin PA5,7 wird als Alternate funktion PP definiert für die SPI kommunikation. PIN PA6 ist für den MISO pin und ist für analog floating.

Port b: pin 1,4,7 sind im output push pull definiert, 1 ist für die Analog psu enable leitung und aktiviert die 5V versorgung. 4 ist für die Reset* leitung. 7 ist für die OPV enable leitung da die opv enleitungen nicht ganz funktionieren immer auf high lassen. PB0 ist open drain ist für den One wire bus für in und out, auf diesem bus sind temp sensoren und auch einen eeprom.

Portb PB8,9,11: die Ports PB8,9 sind open push pull welche auf die anzeige leds geleitet werden, 9 grün, 8 rot. Pin 11 ist für die 3,3V versorgung also immer high.

port c: 0,1,2,3, sind immer gp pp und steuern die relai spulen.

port C: 12 input pull up, ist die ADC* Alarm leitung diese liefert infos wenn ein alarm zustand beim adc auftritt. PC13 ist General purpos pp und ist für den ADC die Reset* leitung.

SPI.c

```
`#ifndef SPI_H #define SPI_H  
  
#include <stm32f10x.h> #include <stdbool.h>  
  
void spi1_init();  
  
void spi1_set_baud(uint8_t baud);  
  
void spi1_rx_dma(bool setting);  
  
uint16_t spi1_transfer16(uint16_t tx_data);  
  
uint8_t spi1_transfer8(uint8_t tx_data);  
  
#endif
```

die funmtnon spi1 init wird die ganze regisdter konfig gesetzt. anfangs wird eine 8bit übertragung gesetz, es wird standart mäßig ein teiler von 16 verwendet. mit der funktion set baud kann der teiler gesetzt werden nach der tabelle:

000:	fPCLK/2
001:	fPCLK/4
010:	fPCLK/8
011:	fPCLK/16
100:	fPCLK/32
101:	fPCLK/64
110:	fPCLK/128
111:	fPCLK/256

die funktion rx_dma kann den dma für die z.b. adc auswertung geladen werden. die funktion spi transfer8 leitet einen transfer ein der daten die in einem uint8 angegeben wird, die funktion transfer 16 verwendet 2 mal die transfer 8 funktion beide funktionen sind blocking.

AD5689

Im AD5689.c wird der AD5689 DAC über SPI angesteuert. Die Initialisierung erfolgt mit AD5689_init(). Dabei wird der DAC zurückgesetzt, der Gain auf 1 gesetzt und der Ausgang zunächst auf 0 V gestellt.

Mit AD5689_send_command() werden die Befehle an den DAC übertragen. Dabei werden insgesamt 24 Bit übertragen. Das erste Byte enthält Befehl und Adresse, die nächsten zwei Bytes den 16-Bit-Datenwert.

Die Funktion AD5689_set_voltage() wandelt eine gewünschte Spannung von 0 V bis 2,5 V in einen 16-Bit-DAC-Wert um. Werte außerhalb dieses Bereichs werden auf 0 V bzw. 2,5 V begrenzt. Der berechnete Wert wird anschließend an den ausgewählten DAC-Kanal übertragen.

Ads8681

Im ADS8681.c wird der ADS8681 ADC über SPI angesteuert.

ADS8681_transfer_frame() überträgt einen 32-Bit Frame. Dieser wird in zwei 16-Bit Übertragungen aufgeteilt. Vor der Übertragung wird ADC_SPI_NCS auf Low gesetzt und danach wieder auf High.

ADS8681_send_Command_blocking() erstellt den benötigten 32-Bit Frame aus Command, Registeradresse und Daten. Der Frame wird anschließend übertragen. Danach wird ein NOP-Frame gesendet, um die Antwort des ADC aus dem Sendepuffer auszulesen.

ADS8681_init() setzt den ADC zuerst über ADC_NRESET zurück. Danach wird über das Range-Select-Register der Eingangsbereich eingestellt. Im Code wird ADS8681_RANGE_SEL_UP_1_25_VREF verwendet.

ADS8681_get_VOLT() wandelt den 16-Bit Rohwert des ADC in eine Spannung um. Dabei wird $V_{REF} = 4,096 \text{ V}$ verwendet. Durch die eingestellte bipolare Eingangsspannung ergibt sich ein Messbereich von ungefähr $-2,56 \text{ V}$ bis $+2,56 \text{ V}$.

Messung am 22.09.2026

Messaufbau mit platine von Drain 1V über $2 \times 8,5 \text{ Meg Ohm}$ widerstände an den Source anschluss. ergebniss sollten:

$I_r = 1\text{V} / 17\text{Meg OHM} = 58.8\text{nA}$ dann am TIA: $U_{out} = I_r * R_f = 58.8\text{nA} * 10\text{Meg} = 0,588\text{V}$

nach dem Dämpfer: $U_d = U_{tia} * (10/33) = 0,178\text{V}$

bei einer messung mit mustimeter kamm richtige spannung bei der adc auswertung kammnen die falschen daten, anschluss messung mit oszi nach dem dämpfer eins mit DC $0,178\text{V}$ überlagertes 400mV pp 50 Hz signal. vermutung aufgrund des widerstands aufbau und einstrahlung da, denn diese einstarhlung $\sim 10\text{meg}$ verstärkt wird ist es sehr groß. *Abschirmung notwendig*.

code für die testung:

```
`int main() { PIN_init(); set_up_uart1(); spi1_init();

PER_33V_PSU_EN = 1; ANALOG_5V_PSU_EN = 1; OPV_PSU_EN = 1;

cal_Select_Reset = 0; cal_Select_set = 0; Range_Select_Reset = 0; Range_Select_set = 0;

DAC_GAIN=0; DAC_SPI_NCS = 1; ADC_SPI_NCS = 1;

uart1_set_baud(9600);

{ uint16_t i; for( i = 0; i < 10000; i++); }

spi1_set_baud(7);

uart_put_string("UART TEST");

set_range_10meg(); reset_GND_Relai();
AD5689_set_Voltage(0,AD5689_ADDR_DAC_AB);
```

```

wait_ms(1000);

static double buffer[1000];

AD5689_set_Voltage(1.0, AD5689_ADDR_DAC_AB); char uart_buffer[64];
wait_ms(1000); int i; for ( i = 0; i < 20; i++) { /* Frame 1: Ergebnis werfen */
ADC_SPI_NCS = 0; spi1_transfer16(0x0000); ADC_SPI_NCS = 1;

for (volatile int j = 0; j < 1000; j++);

/* Frame 2: Ergebnis verwenden */ ADC_SPI_NCS = 0; uint16_t raw =
spi1_transfer16(0x0000); ADC_SPI_NCS = 1;

sprintf( uart_buffer, “%d,0x%04X,%.9f”, i, raw, ADS8681_get_VOLT(raw) );

uart_put_string(uart_buffer); }`

```